

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №6.2

«Быки и коровы»

Выполнил:

студент группы 3825Б1ФИип2
Макарычева А.Д.

Проверил:

Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение.....	3
1. Постановка задачи	4
2. Руководство пользователя	5
3. Руководство программиста.....	6
3.1. Описание структуры программы	6
3.2. Описание структур данных.....	7
3.3. Описание алгоритмов	8-9
4. Результаты экспериментов и тестирования	10-11
Заключение	12
Литература	13

Введение

Предметная область: логические и математические игры. Игра «Быки и коровы»-классическая игра по отгадыванию чисел, развивающая логическое мышление.

Задача: разработать систему классов на языке C++, реализующую игру между человеком и компьютером, где компьютер загадывает число с неповторяющимися цифрами, а игрок пытается его отгадать, получая подсказки в виде количества «быков» (цифр на своих местах) и «коров» (цифр не на своих местах)

1. Постановка задачи

Необходимо разработать программу, реализующую игру «Быки и коровы» со следующими требованиями:

1. Участники: человек (игрок) и компьютер (ведущий)
2. Длина числа: игрок выбирает длину загадываемого числа n (от 1 до 10)
3. Генерация числа: компьютер генерирует n -значное число с неповторяющимися цифрами. Первая цифра не может быть нулем (если $n > 1$)
4. Подсчет результатов: быки (цифры, которые стоят на своих позициях), коровы (цифры, которые присутствуют в загаданном числе, но находятся на других позициях)
5. Завершение игры: игра продолжается, пока игрок не отгадает число полностью (все цифры на своих местах). Программа выводит количество попыток

Пример:

Загадано: 3219

Ввод игрока: 2310

Результат: 1 бык (цифра 1), 2 коровы (цифры 2 и 3)

2. Руководство пользователя

- Запуск программы

После запуска программы пользователь видит приглашение ввести длину числа (от 1 до 10)

```
Enter the length of the desired number (1-10): |
```

- Порядок работы

1. Выбор длины числа. При вводе некорректного значения выводится сообщение об ошибке и запрос повторяется

```
Enter the length of the desired number (1-10): 12
The length should be between 1 and 10!
Enter the length of the desired number (1-10):
```

2. Начало игры. Программа сообщает, что число загадано и начинает поочередно запрашивать варианты игрока

```
Enter the length of the desired number (1-10): 3
The game 'Bulls and Cows' has begun!
The computer made a wish 3-digit number.
Enter 3-digit number with non-repeating digits: |
```

3. Ввод попытки. Игрок вводит число с неповторяющимися цифрами. Длина должна совпадать с выбранной. Если число начинается с нуля (при длине>1) или содержит повторяющиеся цифры, программа сообщит об ошибке и повторит запрос

```
Enter 3-digit number with non-repeating digits: 012
Incorrect input! Try again.
Enter 3-digit number with non-repeating digits: 224
Incorrect input! Try again.
Enter 3-digit number with non-repeating digits: 12
Incorrect input! Try again.
Enter 3-digit number with non-repeating digits: |
```

4. Продолжение игры. Игрок продолжает вводить числа, пока не отгадает задуманное число

```
Enter 3-digit number with non-repeating digits: 672
Result: 3 bulls, 0 cows
Congratulations! You guessed the number for 5 attempts!
```

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из 4 основных классов и main ()

- Класс `NumberGenerator` (генерация случайного числа заданной длины с неповторяющимися цифрами)
- Класс `GameLogic` (проверка корректности ввода и вычисление количества быков и коров)
- Класс `UserInterface` (взаимодействие с пользователем, ввод/ вывод)
- Класс `BullsAndCowsGame` (управление ходом игры)
- `main ()` (главная функция- точка входа в программу, инициализация генератора случайных чисел и запуск игры)

3.2. Описание структур данных

- Класс `NumberGenerator`

Поле `length` (хранит требуемую длину генерируемого числа (задаётся при создании объекта). Допустимые значения: от 1 до 10)

Функция (метод класса) `swap` (вспомогательный метод. Меняет местами значения двух целочисленных переменных, переданных по ссылке)

Функция (метод класса) `shuffle` (вспомогательный метод. Реализует алгоритм для случайного перемешивания элементов вектора)

Функция (метод класса) `NumberGenerator` (конструктор. Инициализирует поле `length` значением `n`)

Функция (метод класса) `generate` (генерирует случайное число и возвращает его в виде строки)

- Класс `GameLogic`

Вложенная структура `BullsAndCows` (хранит результат подсчета)

- Поле `bulls` (количество "быков" — цифр, которые стоят на своих позициях в загаданном числе)
- Поле `cows` (Количество "коров" — цифр, которые присутствуют в загаданном числе, но находятся на других позициях)

Функция (метод класса) `isValidInput` (проверяет корректность введенного числа)

Функция (метод класса) `calculateBullsAndCows` (сравнивает строку-догадку со строкой-секретом и возвращает структуру с количеством быков и коров)

- Класс `UserInterface`

Функция (метод класса) `getLength` (запрашивает у пользователя длину числа (от 1 до 10) с повторным запросом при ошибке)

Функция (метод класса) `getGuess` (запрашивает у пользователя очередную попытку - число заданной длины)

Функция (метод класса) `showResult` (выводит результат)

Функция (метод класса) `showWinMessage` (выводит поздравление с количеством попыток)

- Класс `BullsAndCowsGame`

Поле `length` (длина загаданного числа)

Поле `secretNumber` (строковое представление числа, загаданного компьютером)

Поле `attempts` (счётчик количества попыток, сделанных игроком)

Функция (метод класса) `BullsAndCowsGame` (конструктор. Инициализирует поля `length` и `attempts` (0), затем генерирует секретное число)

Функция (метод класса) `play` (запускает основной игровой цикл)

- `main ()`

`BullsAndCowsGame game(n)` (создание игры с выбранной длиной)

`game.play` (запуск игрового процесса)

3.3. Описание алгоритмов

Алгоритм генерации случайного числа с неповторяющимися цифрами. Метод generate() класса NumberGenerator

```
std::string generate() {
    std::vector<int> digits = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }; // Вектор из цифр от 0 до 9

    shuffle(digits); // Перемешиваем цифры случайным образом

    if (length > 1 && digits[0] == 0) {
        // Ищем ненулевой элемент для замены
        for (int i = 1; i < 10; ++i) {
            if (digits[i] != 0) { // Нашли ненулевую цифру
                swap(digits[0], digits[i]); // Меняем её с первым элементом
                break;
            }
        }
    }

    std::string result; // Строка для хранения результата
    for (int i = 0; i < length; ++i) {
        // Преобразуем цифру в символ и добавляем в строку
        result += static_cast<char>(digits[i] + '0');
    }
    return result;
};
```

Алгоритм для перемешивания цифр (Фишера-Йетса). Метод shuffle() класса NumberGenerator

```
void shuffle(std::vector<int>& digits) {
    // Цикл от последнего элемента до второго
    for (int i = digits.size() - 1; i > 0; --i) {
        int j = rand() % (i + 1); // Генерируем случайный индекс от 0 до i
        swap(digits[i], digits[j]); // Меняем местами элементы i и j
    }
}
```

Алгоритм проверки корректности ввода. Метод isValidInput() класса GameLogic

```
static bool isValidInput(const std::string& input, int length) {
    // Проверяем, что длина введенной строки равна заданной
    if (input.length() != length) return false;

    // Проверяем, что все символы – цифры
    for (char c : input) { // Перебираем каждый символ строки
        if (!std::isdigit(c)) return false; // Если не цифра – возвращаем false
    }

    // Проверяем, что число не начинается с нуля (если длина > 1)
    if (length > 1 && input[0] == '0') {
        return false; // Первая цифра не может быть 0
    }

    // Вектор для отслеживания использованных цифр (индексы 0-9)
    std::vector<bool> seen(10, false);
    // Проверяем, что все цифры уникальны
    for (char c : input) {
        int digit = c - '0'; // Преобразуем символ в число
        if (seen[digit]) return false; // Если цифра уже встречалась – ошибка
        seen[digit] = true; // Отмечаем цифру как использованную
    }
    return true; // Все проверки пройдены успешно
}
```


Алгоритм подсчета быков и коров. Метод calculateBullsAndCows() класса GameLogic

```
static BullsAndCows calculateBullsAndCows(const std::string& guess, const std::string& secret) {
    BullsAndCows result{ 0, 0 };
    int length = secret.length(); // Получаем длину чисел

    // Векторы для отметки уже использованных позиций
    std::vector<bool> secretUsed(length, false); // Отмечаем использованные позиции в тайном числе
    std::vector<bool> guessUsed(length, false); // Отмечаем использованные позиции в попытке

    // Первый проход: ищем быков (точные совпадения)
    for (int i = 0; i < length; ++i) {
        if (guess[i] == secret[i]) {
            result.bulls++;
            secretUsed[i] = true; // Отмечаем позицию в тайном числе как использованную
            guessUsed[i] = true; // Отмечаем позицию в попытке как использованную
        }
    }

    // Второй проход: ищем коров (цифры есть, но не на своих местах)
    for (int i = 0; i < length; ++i) {
        if (guessUsed[i]) continue; // Пропускаем уже использованные позиции из попытки
        // Ищем такую же цифру в тайном числе
        for (int j = 0; j < length; ++j) {
            if (!secretUsed[j] && guess[i] == secret[j]) { // Если позиция в тайном числе не использована и цифры совпадают
                result.cows++;
                secretUsed[j] = true; // Отмечаем позицию в тайном числе как использованную
                break;
            }
        }
    }

    return result;
}
```

Алгоритм основного игрового цикла. Метод play() класса BullsAndCowsGame

```
void play() {
    std::cout << "The game 'Bulls and Cows' has begun!\n";
    std::cout << "The computer made a wish " << length << "-digit number.\n";

    // Бесконечный цикл (выход при победе)
    while (true) {
        attempts++;
        // Получаем попытку от пользователя
        std::string guess = UserInterface::getGuess(length);
        // Вычисляем быков и коров
        GameLogic::BullsAndCows result = GameLogic::calculateBullsAndCows(guess, secretNumber);

        UserInterface::showResult(result); // Показываем результат

        if (result.bulls == length) { // Если количество быков равно длине числа (все цифры угаданы)
            UserInterface::showWinMessage(attempts); // Показываем сообщение о победе
            break; // Выходим из цикла (игра окончена)
        }
    }
}
```

4. Результаты экспериментов и тестирования

Корректная работа при n=1

```
Enter the length of the desired number (1-10): 1
The game 'Bulls and Cows' has begun!
The computer made a wish 1-digit number.
Enter 1-digit number with non-repeating digits: 0
Result: 0 bulls, 0 cows
Enter 1-digit number with non-repeating digits: 1
Result: 0 bulls, 0 cows
Enter 1-digit number with non-repeating digits: 2
Result: 0 bulls, 0 cows
Enter 1-digit number with non-repeating digits: 3
Result: 0 bulls, 0 cows
Enter 1-digit number with non-repeating digits: 4
Result: 0 bulls, 0 cows
Enter 1-digit number with non-repeating digits: 5
Result: 1 bulls, 0 cows
Congratulations! You guessed the number for 6 attempts!
```

Ошибка длины

```
Enter the length of the desired number (1-10): 4
The game 'Bulls and Cows' has begun!
The computer made a wish 4-digit number.
Enter 4-digit number with non-repeating digits: 123
Incorrect input! Try again.
Enter 4-digit number with non-repeating digits: |
```

Ошибка: первая цифра 0

```
Enter the length of the desired number (1-10): 6
The game 'Bulls and Cows' has begun!
The computer made a wish 6-digit number.
Enter 6-digit number with non-repeating digits: 012345
Incorrect input! Try again.
Enter 6-digit number with non-repeating digits: |
```

Ошибка: повтор цифр

```
Enter the length of the desired number (1-10): 5
The game 'Bulls and Cows' has begun!
The computer made a wish 5-digit number.
Enter 5-digit number with non-repeating digits: 11234
Incorrect input! Try again.
Enter 5-digit number with non-repeating digits: |
```

Ошибка: не цифра

```
Enter the length of the desired number (1-10): 4
The game 'Bulls and Cows' has begun!
The computer made a wish 4-digit number.
Enter 4-digit number with non-repeating digits: 12a3
Incorrect input! Try again.
Enter 4-digit number with non-repeating digits:
```

Правильный подсчет быков и коров, количества попыток

```
Enter the length of the desired number (1-10): 3
The game 'Bulls and Cows' has begun!
The computer made a wish 3-digit number.
Enter 3-digit number with non-repeating digits: 123
Result: 1 bulls, 0 cows
Enter 3-digit number with non-repeating digits: 145
Result: 2 bulls, 0 cows
Enter 3-digit number with non-repeating digits: 147
Result: 1 bulls, 1 cows
Enter 3-digit number with non-repeating digits: 157
Result: 1 bulls, 2 cows
Enter 3-digit number with non-repeating digits: 175
Result: 3 bulls, 0 cows
Congratulations! You guessed the number for 5 attempts!
```

Заключение

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. Разработана система классов, реализующая игру «Быки и коровы»
2. Реализована генерация чисел с неповторяющимися цифрами
3. Реализована полная логика подсчёта быков и коров
4. Обеспечена обработка ошибок ввода пользователя
5. Программа протестирована на различных наборах данных, все тесты пройдены

Литература

1. Сайт cppreference: <https://en.cppreference.com/>